

软件过程改进方案设计文档

作业编号: Assgt_02-2

提交人: 2023141461016-苏麟俊峰

提交日期: 2026-06-30

基于项目: 学生管理系统 (Java MVC + Servlet + JSP + MySQL)

1. 改进方案概述

本方案以 ISO/IEC 12207 软件生命周期过程标准为框架，在第一次作业已定义的软件过程基础上，系统性地将 AI 技术融入需求分析、系统设计、编码实现、测试验证、运维监控五大环节，构建「人机协同」的新型软件过程体系。

改进目标：

- 在不增加团队人力负担的前提下，提升各阶段产出质量与效率
- 保留第一次作业中已裁剪适配小团队的过程特征，确保可落地
- 为 AI 输出物建立可验证的质量标准，避免"黑箱交付"

2. 新增 AI 相关角色

在原团队角色（项目经理、后端开发、前端开发、测试人员）基础上，新增以下 AI 相关角色：

角色	职责	对应原角色
AI 训练师	编写和优化 Prompt 模板；管理 AI 工具的配置与版本；评估 AI 输出质量并根据反馈调优	可由后端开发兼任
AI 测试分析师	使用 AI 工具生成测试用例并审查其有效性；分析 AI 测试覆盖率报告；标记 AI 误判的缺陷	可由测试人员兼任
AI 输出审核员	对所有 AI 生成的关键交付物（代码、文档、测试用例）进行人工确认和质量把关	可由项目经理兼任

设计原则：不增加编制——新角色全部由现有团队成员兼任，AI 工具承担的是"产能倍增器"角色而非"替代人"角色。

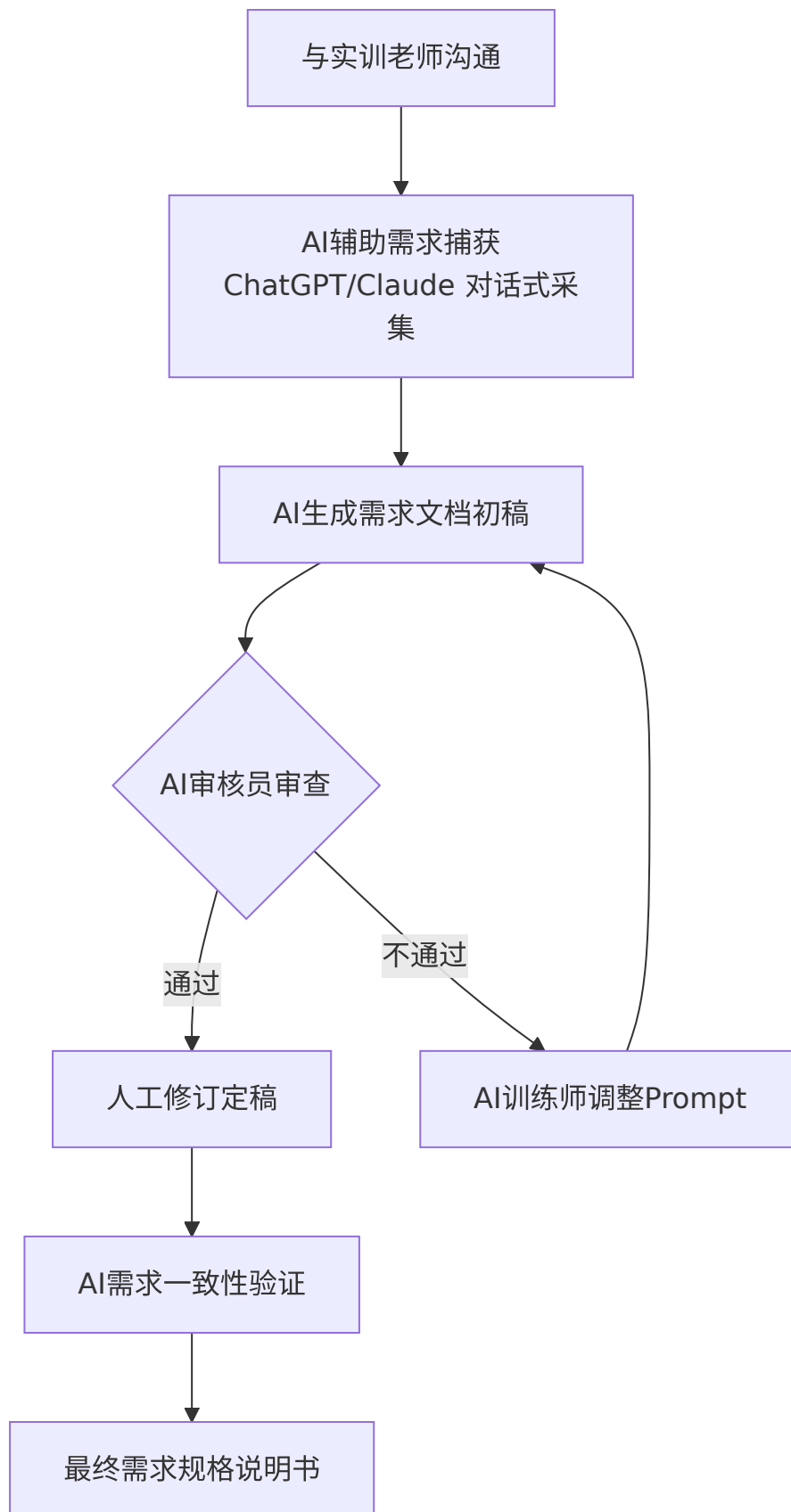
3. 流程改进

3.1 主过程改进（含 AI 参与节点）

3.1.1 获取过程（需求分析）

原有流程：与实训老师沟通 → 查阅同类系统 → 编写需求文档

改进后流程：



AI 参与节点：

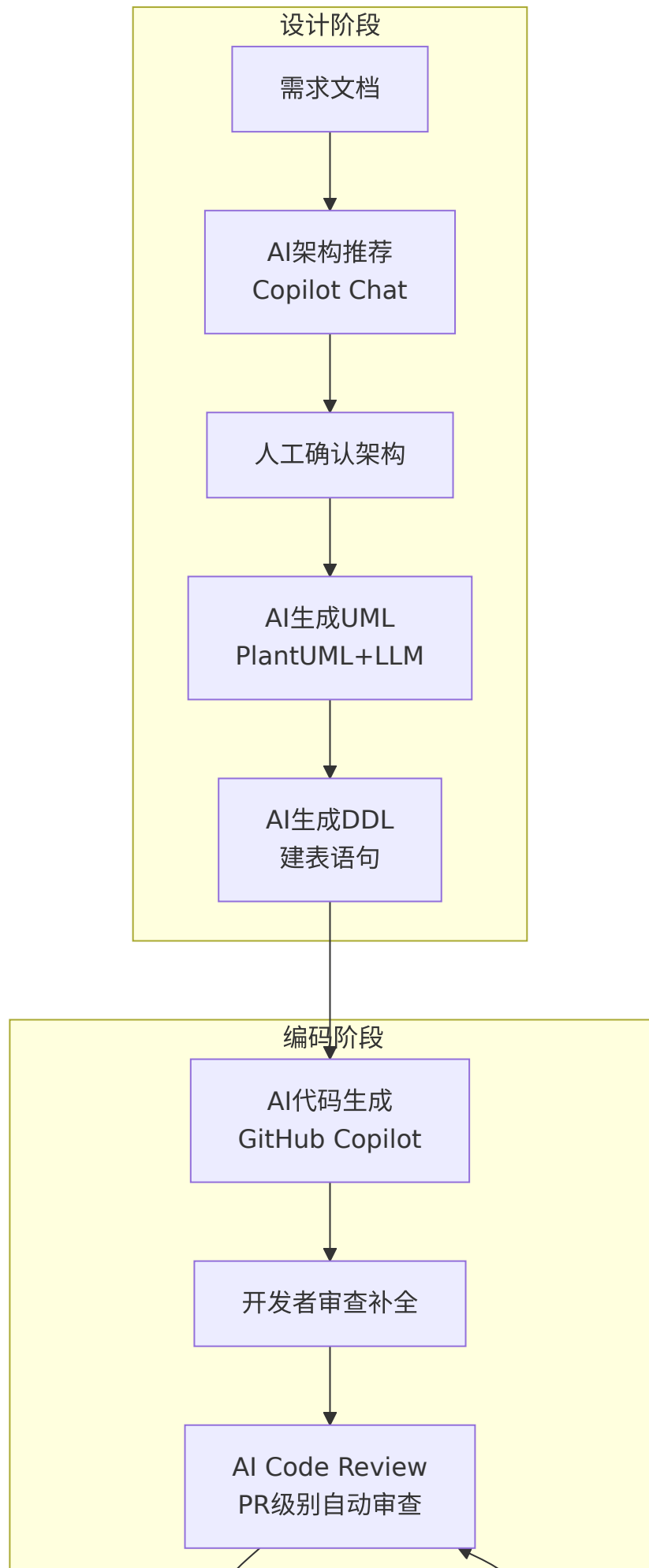
- **节点 B：**使用 ChatGPT 将老师口头需求转化为结构化需求条目
- **节点 C：**自动生成符合标准格式的需求文档初稿

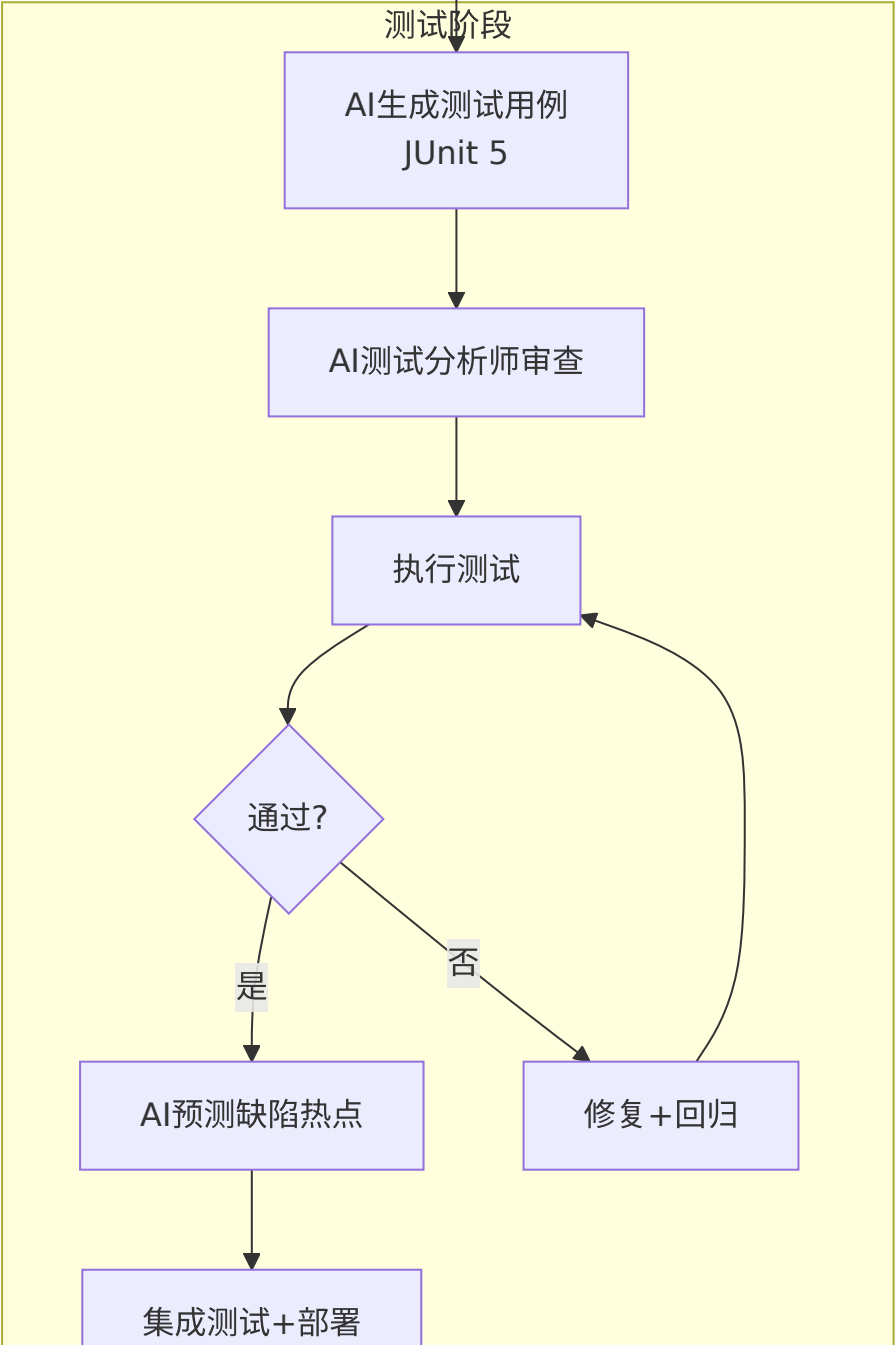
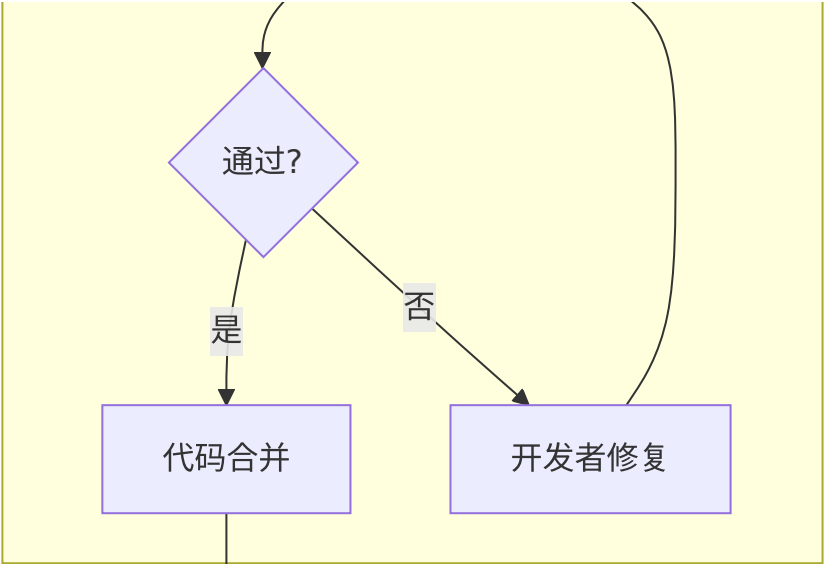
- **节点 G：** LLM 语义分析，检测需求间矛盾、遗漏、模糊表述

3.1.2 开发过程（设计 + 编码 + 测试）

原有流程： 设计架构 → 设计数据库 → 编写代码 → 单元测试 → 集成测试 → 部署

改进后流程：





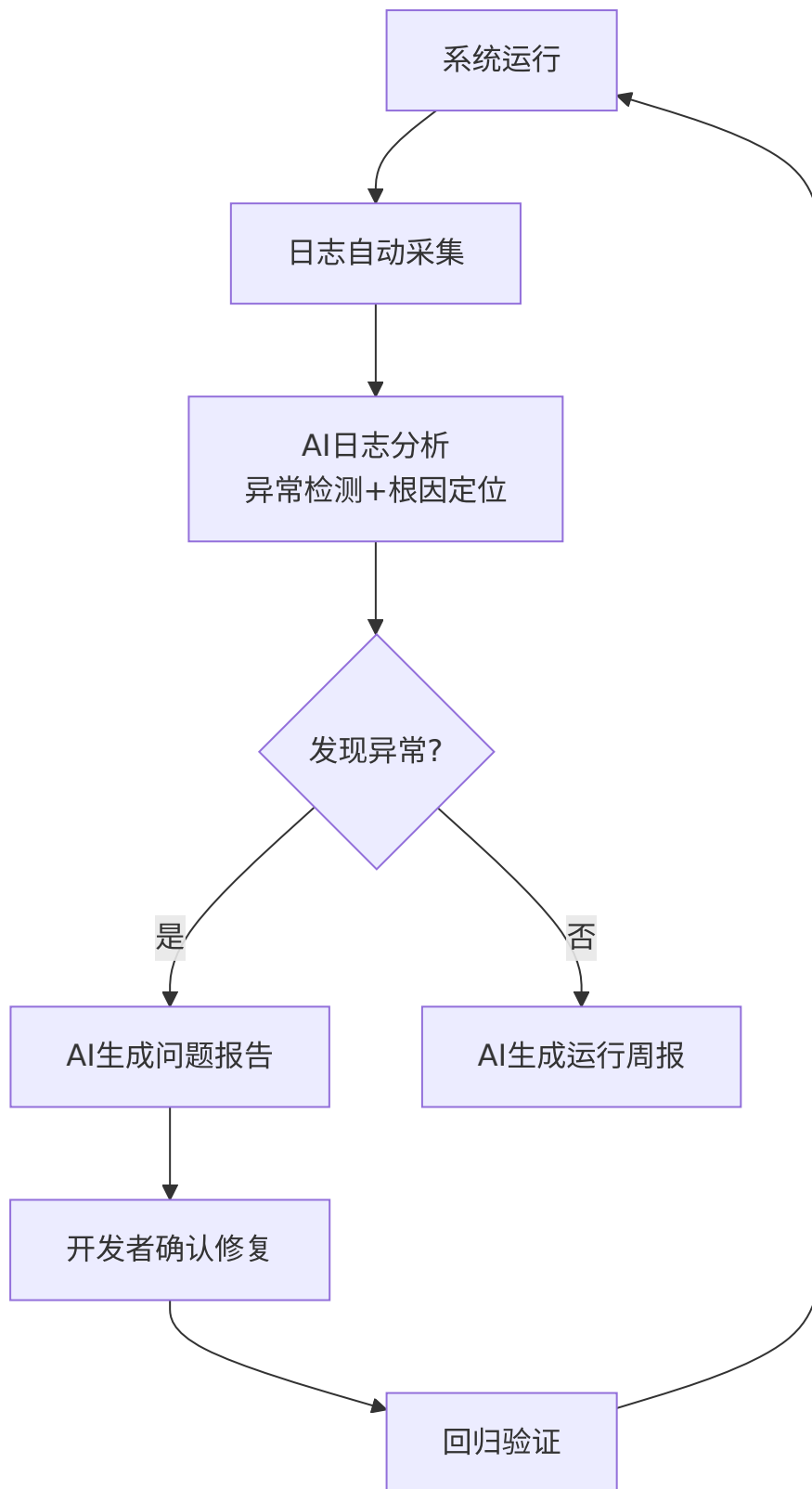


AI 参与节点：

- **A2**：输入需求摘要，Copilot Chat 推荐 MVC 各层组件划分
- **A4**：从 Java 实体类反向生成类图，从 DDL 生成 ER 图
- **B1**：生成 DAO 层 CRUD 代码、Service 层业务模板
- **B3**：PR 提交时自动运行 AI Code Review，检查 SQL 注入、空指针、资源泄漏
- **C1**：读取 Service 方法签名，自动生成边界值/异常路径测试用例
- **C5**：基于代码变更热力图，预测高缺陷概率模块

3.1.3 运行与维护过程

改进后流程：



AI 参与节点：

- **C**：Tomcat 日志导入 LLM，自动识别异常堆栈根因和频率
- **H**：自动汇总一周运行数据，生成可读的运行报告

3.2 支持过程改进

原过程	改进内容	AI 工具
文档过程	AI 辅助生成需求文档、设计文档、用户手册初稿；人工修订定稿	ChatGPT / Claude
配置管理过程	AI 辅助生成 Git Commit Message（Conventional Commits 格式）；AI 分析分支冲突原因	GitHub Copilot / ChatGPT
质量保证过程	AI Code Review 替代人工初审；缺陷热力图指导测试资源分配	GitHub Copilot / SonarQube AI
验证过程	AI 生成测试用例补充人工遗漏的边界场景	ChatGPT / Diffblue Cover
确认过程	AI 分析用户测试反馈，自动归类问题类型并生成优先级排序	ChatGPT
联合评审过程	AI 自动生成会议纪要和阶段评审报告摘要	ChatGPT / Claude
问题解决过程	AI 辅助根因分析；自动推荐相似历史问题的解决方案	ChatGPT + 知识库检索

3.3 组织过程改进

原过程	改进内容
管理过程	AI 辅助生成周报和进度跟踪报告；AI 风险评估（基于任务复杂度+成员负载）
基础设施过程	AI 辅助生成 Dockerfile、CI/CD 配置文件
改进过程	AI 分析过程度量数据，自动识别瓶颈并提出改进建议
人力资源过程	AI 根据成员技能标签和任务需求自动推荐任务分配方案

4. 工具链选型

阶段	推荐工具	核心功能	集成方式	预期效率提升
需求分析	ChatGPT 4o / Claude 4	需求捕获、文档生成、一致性验证	Web 端直接使用 + Prompt 模板库	文档编写时间 -50%

系统设计	GitHub Copilot Chat + PlantUML	架构推荐、UML 生成	IDE 插件 + PlantUML 在线渲染	设计图绘制时间 -40%
编码实现	GitHub Copilot	代码补全、方法生成、注释转代码	VS Code / IntelliJ IDEA 插件	编码效率 +35%
代码审查	GitHub Copilot Code Review	PR 级别自动审查	GitHub 原生集成	审查问题发现率 +50%
测试生成	ChatGPT 4o / Copilot	JUnit 5 测试用例生成	IDE 内 Prompt 调用	测试编写时间 -65%
运维监控	ChatGPT (日志分析)	异常检测、根因定位	日志文件导入分析	问题定位时间 -45%

5. AI 输出物质量标准

AI 输出物类型	质量要求	验证机制
需求文档	需求条目覆盖率 ≥ 95%；无可检测矛盾	人工逐条核对 + LLM 一致性自动检查
UML 图	类/关系无遗漏；与代码结构一致	从代码反向生成 UML 进行比对
AI 生成代码	编译通过率 100%；无安全红线问题	CI 自动编译 + SonarQube 扫描
AI 生成测试用例	可执行率 ≥ 90%；至少覆盖 2 个边界/异常场景	执行后检查通过率和覆盖率
AI Code Review 结论	误报率 ≤ 20%	人工抽样复核 30% 的 Review 结论
AI 生成文档	事实准确率 100%；格式符合模板	人工全文审校

核心原则：所有 AI 输出物必须经过人工确认方可作为正式交付物——AI 是"建议者"，人是"决策者"。

6. 风险评估与应对策略

风险类别	风险描述	严重程度	应对策略
数据安全	将项目代码/需求上传至云端 LLM 存在泄露风险	高	① 敏感数据（数据库密码、用户信息）不上传 AI 工具 ② 优先使用本地部署模型（如 Qwen/Llama）处理敏感内容 ③ 上传前脱敏处理
模型偏见	LLM 训练数据偏差导致生成代码/文档存在隐藏缺陷	中	① 人工审查所有 AI 输出 ② 对 AI 推荐的设计模式/架构进行团队讨论确认 ③ 建立"AI 输出审查清单"
技术依赖	团队过度依赖 AI 工具，人工能力退化	中	① 要求所有成员能解释 AI 生成代码的逻辑 ② 每周安排"无 AI 编码"练习 ③ Code Review 时必须审查对 AI 代码的理解程度
质量不可控	AI 生成内容质量波动大，同一 Prompt 不同时间产出差异显著	中	① 建立 Prompt 模板库实现提示词标准化 ② 设置 AI 输出物的最低可接受标准 ③ 对关键交付物使用多次生成取最优策略
幻觉问题	LLM 可能生成看似合理但实际错误的代码或文档内容	中	① 要求 AI 生成代码必须通过编译和单元测试 ② 文档中的事实性陈述必须可追溯至源材料 ③ 使用 RAG（检索增强生成）降低幻觉率
成本控制	API 调用费用随使用量增长	低	① 优先使用免费/教育版工具（GitHub Copilot 学生包） ② 批量处理减少 API 调用次数 ③ 评估本地开源模型替代方案

7. 过程度量指标（含 AI 贡献度）

指标	计算方式	目标值
AI 代码生成采纳率	AI 生成代码中被人工接受并入仓的比例	≥ 70%
AI 测试覆盖率贡献	AI 生成测试用例带来的覆盖率增量	≥ 20 个百分点
AI Code Review 检出率	AI 发现的有效问题数 / 总问题数	≥ 60%
文档编写效率提升	(原耗时 - AI 辅助耗时) / 原耗时	≥ 40%

AI 输出人工修订率	人工修改的 AI 输出行数 / AI 输出总行数	≤ 30%（越低越好）
------------	--------------------------	-------------
